COSC 461/561 Project 1: Eagle Delivery

Part A: A* Search

(a) Problem formulation

- State: the robot's position as (row, column).
- Initial state: the start building, for example S = (1, 1).
- Actions: UP, DOWN, LEFT, RIGHT.
- Transition model: the robot moves one cell. If the cell is a wall or outside the map, it stays where it is.
- Goal test: the robot is at the destination building.
- Path cost: each move costs 1.

BFS is optimal here because every move costs the same (1). BFS looks at all routes of length 1, then length 2, and so on, so the first route it finds to the goal is the shortest one.

(b) Manhattan distance. `manhattan` returns |row difference| + |column difference|. It is admissible because the robot can only move one row or one column per step, so it needs at least this many moves. Walls can only make the route longer, so the heuristic never overestimates.

(c) A* implementation. I used `heapq` with entries (f, insertion number, g, state), where f = g + h. The insertion number breaks ties. I kept two dictionaries, `g_costs` and `parents`. If a popped entry has an old g value, I skip it. I check for the goal when an entry is popped, and I count a node as expanded when I generate its neighbors. If a cheaper route to a state is found, I update its cost and parent and push it again.

Map	Task	BFS length	A* length	BFS expanded	A* expanded
Baseline	S -> W	18	18	67	22
Baseline	L -> D	22	22	79	62
Baseline	H -> P	16	16	41	16
Detour	S -> W	26	26	70	51
Detour	L -> D	22	22	65	60
Detour	H -> P	16	16	41	16

BFS and A* found routes of the same length every time, since both are optimal. A* always expanded fewer nodes, but not by the same amount. For H -> P it only expanded the 16 cells on the route, because the route goes almost straight to P. For L -> D the difference was small, because the walls make the robot go away from D first, so the heuristic is not very helpful there. On the detour map, S -> W became 8 steps longer and A* expanded more nodes (51 instead of 22) because the closed cell blocks the path that looks shortest.

Part B: Q-Learning

Implementation. In `epsilon_greedy`, if `rng.random()` is less than epsilon, I pick a random action. Otherwise I pick the action with the highest Q-value (unseen = 0.0), and if there is a tie I pick randomly among the tied actions. In `q_learning_update`, the target is the reward if the episode ended, otherwise reward + gamma × the best Q-value of the next state. Then Q(s, a) = Q(s, a) + alpha × (target − Q(s, a)).

Settings: 2000 training episodes, 100 evaluation episodes, seeds 0, 1, 2, alpha = 0.3, gamma = 0.95, epsilon = 0.2, slip = 0, max 200 steps.

Task	Eval success rate	Eval steps	BFS shortest	Avg training steps (first 100 episodes)	Avg training steps (last 100 episodes)
S -> W	100%	18	18	98	23
L -> D	100%	22	22	94	28
H -> P	100%	16	16	58	20

Observations.

- In evaluation, all three seeds delivered the package every time on every task, using the shortest route. So Q-learning learned the best route without using the map.
- The agent learned quickly. In the first 100 episodes it sometimes ran out of steps, but after about 200 episodes it succeeded almost every time.
- Training episodes were still a bit longer than the shortest route at the end, because 20% of the actions are random. In evaluation epsilon is 0, so this goes away.
- All three seeds gave the same evaluation results. H -> P was the easiest task and L -> D was the hardest.



Average training steps per episode for the three tasks (average of 3 seeds). After episode 600 the lines stay flat.

Part C: Custom Map and Experiment

(a) My map. `maps/my_campus.txt` is 12 x 19 and uses each building letter once.

```
###################
#S......#........W#
#.#####.#.#######.#
#.#...#.#.#.....#.#
#.#.#.#...#.###.#.#
#...#.#####.#H#...#
###.#.......#.#####
#L#.#####.###.....#
#.#.....#...#.###.#
#.#####.###.#.#D#.#
#.......#P....#...#
###################
```

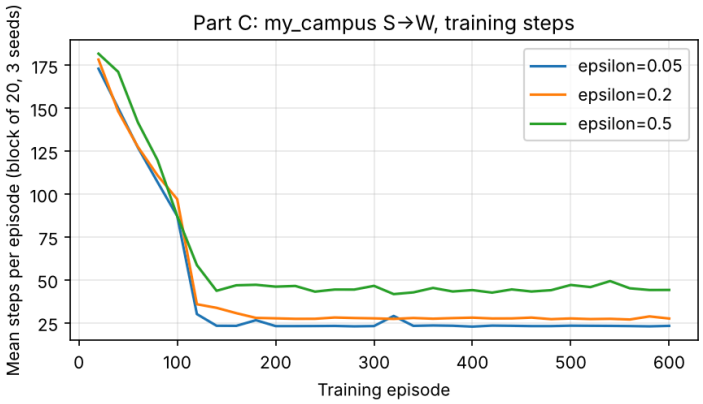
There is one loop that goes through S and W, so there are two ways to get to W: a short route along the top (22 steps) and a longer route through the middle (34 steps). There are also dead-end branches: one going down to L, and one going down to P, D, and H. On the S -> W task, BFS found length 22 with 65 expanded nodes, and A* found length 22 with 25 expanded nodes.

(b) Question and prediction. How does epsilon affect learning and the final result? I tested epsilon = 0.05, 0.2, and 0.5. I predicted that a higher epsilon would explore more and find the short route more reliably, but would have longer training episodes. I also thought epsilon = 0.05 might get stuck using the long route.

(c) Method. I only changed epsilon. Everything else stayed the same as Part B: the S -> W task on my map, 2000 training episodes, 100 evaluation episodes with epsilon = 0, seeds 0, 1, 2, alpha = 0.3, gamma = 0.95, slip = 0,

max 200 steps, and a new Q-table for each run.

Epsilon	Eval success rate	Eval steps	Avg training steps (last 100 episodes)
0.05	100%	22	23
0.2	100%	22	27
0.5	100%	22	44



Average training steps per episode on my map for each epsilon (average of 3 seeds).

(d) Results and discussion. All three epsilon values ended with the same result: every seed used the short 22-step route and succeeded every time. So epsilon made no difference to the final performance, and my guess that 0.05 would get stuck was wrong. I think this is because every step costs −1 and untried actions start at 0, so untried actions look better than tried ones. Because of that, the agent still explores early on even with a small epsilon.

Epsilon did make a difference during training. With epsilon = 0.5, training episodes stayed around 44 steps, about double the shortest route, because half the moves are random. With epsilon = 0.05, they were close to the shortest route (23 steps). So on this small map, a small epsilon was better: the same final result with less wasted training. A higher epsilon might matter more on a bigger map or with slip.